

1 Notation and Symbols

1.1 Variables and Sets

t	Discrete time step index ($t = 0, 1, 2, \dots$)
S_t	State at time t (random variable, uppercase)
s, s'	Particular state values (lowercase); s' denotes a next state
A_t	Action at time t (random variable)
a, a'	Particular action values
R_t	Reward received at time t (random variable)
r	A particular reward value
G_t	Return (cumulative discounted reward) from time t onward
T	Terminal time step (episodic tasks)
γ	Discount factor, $0 \leq \gamma \leq 1$
α	Step-size / learning rate parameter
θ	Convergence threshold for iterative algorithms
Δ	Maximum value change across a sweep (used in DP algorithms)
k	Iteration index in iterative algorithms
$\mathcal{S}$	Set of all states
$\mathcal{S}^+$	Set of all states including terminal state(s)
$\mathcal{A}$	Set of all actions
$\mathcal{A}(s)$	Set of actions available in state s
$\mathcal{R}$	Set of all possible reward values, $\mathcal{R} \subset \mathbb{R}$
$ \mathcal{S} $	Number of states (cardinality of $\mathcal{S}$)
$ \mathcal{A} $	Number of actions

1.2 Policies

π	A policy (mapping from states to action probabilities)
$\pi(a s)$	Probability of taking action a in state s under policy π
$\pi(s)$	The action selected by a deterministic policy in state s
π_*	An optimal policy
π'	An improved policy (e.g., after greedy improvement)

1.3 Value Functions

$v_\pi(s)$	State-value function: expected return starting from s , following π
$q_\pi(s, a)$	Action-value function: expected return starting from s , taking a , then following π
$v_*(s)$	Optimal state-value function: $\max_\pi v_\pi(s)$
$q_*(s, a)$	Optimal action-value function: $\max_\pi q_\pi(s, a)$
$V(s)$	An estimate or array approximating a value function (mutable)
$V_k(s)$	Value estimate at iteration k

1.4 Probability and Dynamics Notation

$\mathbb{E}_\pi[\cdot]$	Expected value under policy π (agent follows π)
$\mathbb{E}_\pi[\cdot S_t = s]$	Conditional expectation given state s at time t , following π
$p(s', r s, a)$	Dynamics function: $\Pr\{S_t = s', R_t = r S_{t-1} = s, A_{t-1} = a\}$ Joint probability of next state s' and reward r , given current s and a
$p(s' s, a)$	State-transition probability (dynamics marginalized over r)
$r(s, a)$	Expected reward for state-action pair
$r(s, a, s')$	Expected reward for state-action-next-state triple

1.5 Operators and Common Notation

$\sum_a$	Shorthand for $\sum_{a \in \mathcal{A}(s)}$
$\sum_{s', r}$	Shorthand for $\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}}$
$\max_a$	Maximize over all actions $a \in \mathcal{A}(s)$
$\arg \max_a$	The action a that achieves the maximum
$\leftarrow$	Assignment (update a stored value)
$\geq$ (on policies)	$\pi \geq \pi'$ means $v_\pi(s) \geq v_{\pi'}(s)$ for all s

2 Temporal-Difference (TD) Learning

Basic TD update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)] \quad (1)$$

where α is the learning rate (step size).

TD combines ideas from Monte Carlo (learning from experience) and Dynamic Programming (bootstrapping from estimates).

3 Finite Markov Decision Processes

3.1 Agent-Environment Interface

Agent and environment interact in discrete time steps: $t = 0, 1, 2, 3, \dots$

Trajectory: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$

At each time t :

- Agent observes state $S_t \in \mathcal{S}$
- Selects action $A_t \in \mathcal{A}(s)$
- Receives reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$
- Transitions to next state S_{t+1}

3.2 Dynamics Function

Complete probability distribution defining MDP dynamics:

$$p(s', r | s, a) = \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \quad (2)$$

for all $s', s \in \mathcal{S}, r \in \mathcal{R}, a \in \mathcal{A}(s)$.

Must satisfy: $\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1$ for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$.

3.3 Derived Quantities

State-transition probabilities:

$$p(s' | s, a) = \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a) \quad (3)$$

Expected reward for state-action pair:

$$r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a) \quad (4)$$

Expected reward for state-action-next-state triple:

$$r(s, a, s') = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)} \quad (5)$$

3.4 Markov Property

Markov Property: The future depends only on the current state, not on the history.

$$\Pr\{S_t = s', R_t = r \mid S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}\} = p(s', r | S_{t-1}, A_{t-1}) \quad (6)$$

A state signal has the Markov property if it includes all relevant information from history.

3.5 Returns

Episodic tasks (terminate in finite steps at terminal state):

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T \quad (7)$$

where T is the terminal time step.

Continuing tasks (no terminal state, with discounting):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad (8)$$

where $0 \leq \gamma \leq 1$ is the discount rate.

Recursive form:

$$G_t = R_{t+1} + \gamma G_{t+1} \quad (9)$$

Special case (constant reward +1 forever):

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma} \quad (10)$$

3.6 Policies

Policy π : Mapping from states to probability distributions over actions.

$$\pi(a | s) = \Pr\{A_t = a \mid S_t = s\} \quad (11)$$

Deterministic policy: $\pi(s) = a$ (single action for each state).

3.7 Value Functions

State-value function for policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right] \quad (12)$$

Action-value function (Q-function) for policy π :

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right] \quad (13)$$

Relationship between value functions:

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) q_{\pi}(s, a) \quad (14)$$

3.8 Bellman Equations

Bellman equation for v_π :

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')] \quad (15)$$

Expanded form:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \quad (16)$$

Bellman equation for q_π :

$$q_\pi(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a') \right] \quad (17)$$

Expanded form:

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \quad (18)$$

3.9 Optimal Value Functions

Optimal state-value function:

$$v_*(s) = \max_\pi v_\pi(s) \quad \text{for all } s \in \mathcal{S} \quad (19)$$

Optimal action-value function:

$$q_*(s, a) = \max_\pi q_\pi(s, a) \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A}(s) \quad (20)$$

Relationship:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (21)$$

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (22)$$

3.10 Bellman Optimality Equations

Bellman optimality equation for v_* :

$$v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (23)$$

Bellman optimality equation for q_* :

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} q_*(s', a') \right] \quad (24)$$

3.11 Optimal Policy

Optimal policy (greedy with respect to q_*):

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (25)$$

Alternative form (greedy with respect to v_* using one-step lookahead):

$$\pi_*(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (26)$$

Any policy that is greedy with respect to v_* or q_* is an optimal policy.

4 Dynamic Programming

DP methods require perfect model of environment: $p(s',r|s,a)$.

4.1 Policy Evaluation (Prediction)

Goal: Compute v_π for a given policy π .

Iterative update (until convergence):

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')] \quad (27)$$

for all $s \in \mathcal{S}$.

Convergence criterion: Stop when $\max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \theta$ for small threshold $\theta > 0$.

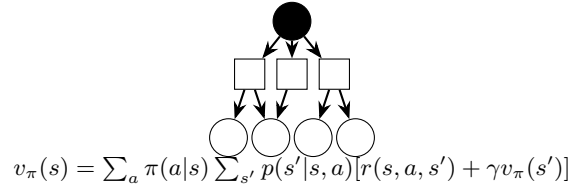
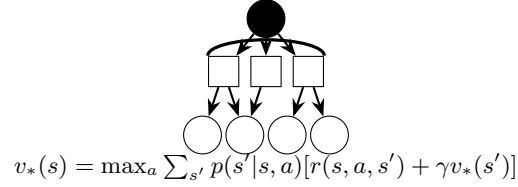
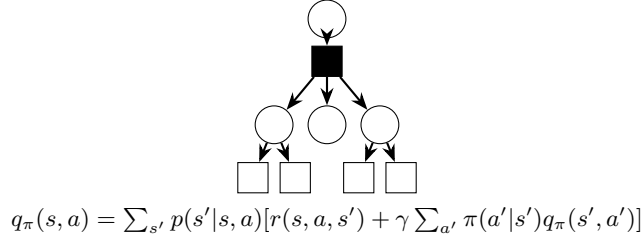
As $k \rightarrow \infty$, $V_k \rightarrow v_\pi$ under Bellman equation constraints.

Implementation variants:

- **Two-array (Jacobi):** Use old values for all states in sweep
- **In-place (Gauss-Seidel):** Use new values as soon as available; typically faster convergence

Algorithm 1: Iterative Policy Evaluation**Input:** Policy π , threshold $\theta > 0$ Initialize $V(s) = 0$ for all $s \in \mathcal{S}^+$ **repeat** $\Delta \leftarrow 0$ **for each** $s \in \mathcal{S}$ **do** $v \leftarrow V(s)$ $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$ $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ **end for****until** $\Delta < \theta$ **Output:** $V \approx v_\pi$ **4.2 Policy Improvement****Policy improvement theorem:** Let π and π' be deterministic policies such that for all $s \in \mathcal{S}$:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad (28)$$

Then $v_{\pi'}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$.**5 Backup Diagrams****5.1 v_π Backup Diagram****5.2 v_* Backup Diagram****5.3 q_π Backup Diagram****5.4 q_* Backup Diagram**